Análisis del Flujo SDLC IOND

Estados, responsables, puntos de control y frontera de roles

Propuesta por: Steve Nina — Scrum Master IOND
Aplica a: IOND · IonMind · StageMind
Audiencia: PO · Lead Dev · RM · Lead QA · Scrum Master

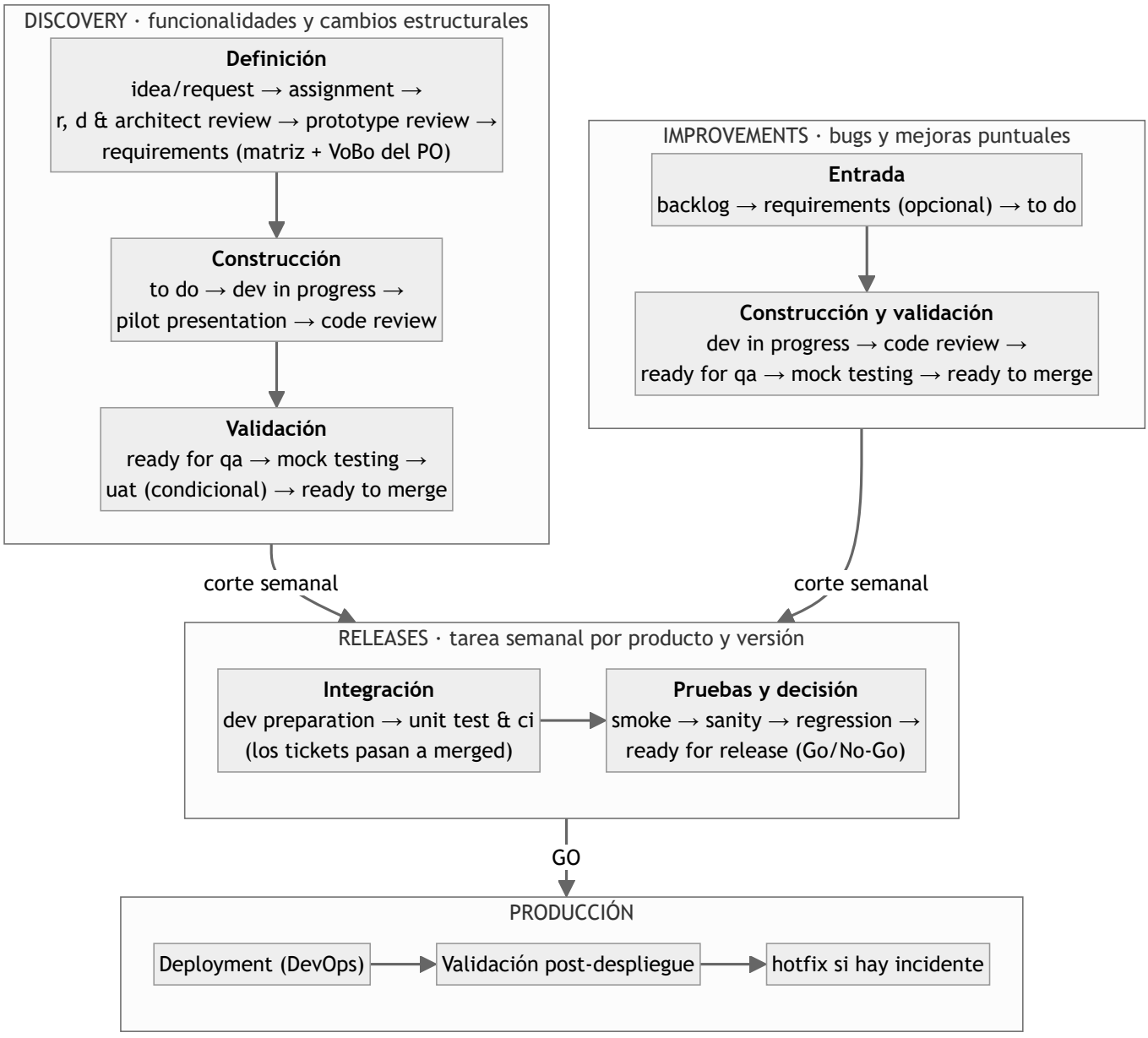
1. Propósito

Este documento analiza el flujo SDLC de IOND estado por estado y propone su forma consolidada. Para cada estado define **qué se espera, quién responde, quién mueve el ticket, qué se exige para entrar y qué evidencia se deja al salir**. Incluye los diagramas del flujo, la matriz RACI y el **contrato de frontera entre QA, PO y Development**, que es el punto más sensible del nuevo rol de QA como Analista de Calidad y Requerimientos.

- La **fuente canónica** de los detalles operativos (fichas completas, campos, tags, plantillas y automatizaciones) es `knowledge/L1-project/sdlc-consolidation.md`. El anexo de este documento la reproduce.
- Cada regla remite a su decisión **Q-xx** del documento *03 — Registro de Decisiones, Riesgos y Limitaciones*.

---

2. Vista general del SDLC IOND



Lectura del modelo:

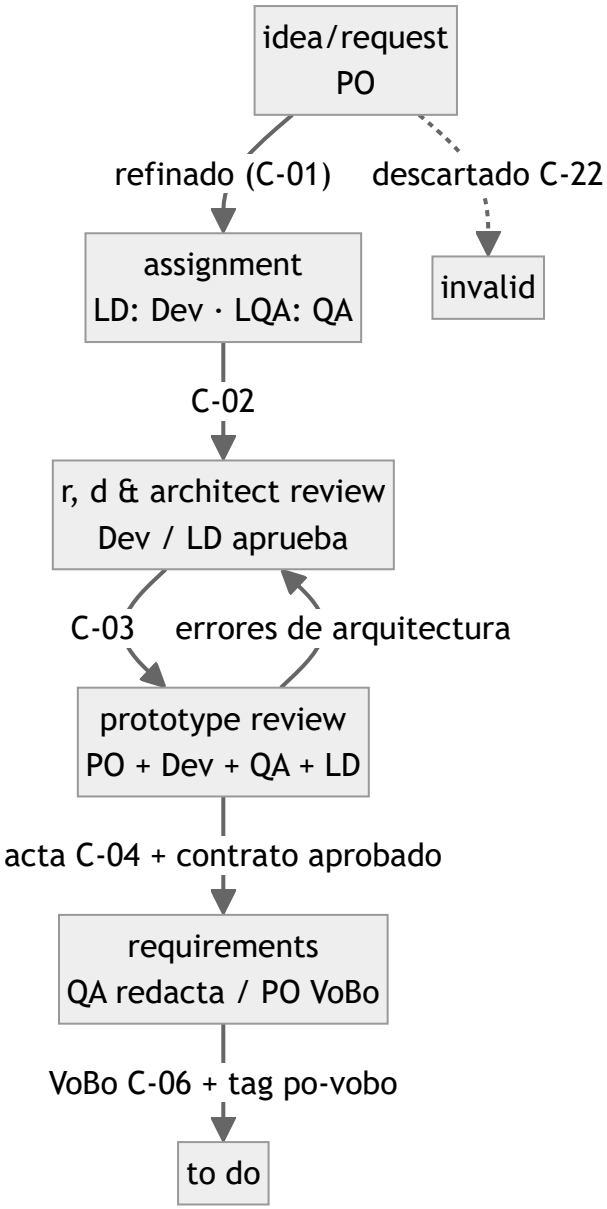
1. **Discovery** lleva una funcionalidad desde la idea hasta que queda lista para integrarse. Incluye diseño (R&D y prototipo), definición de requerimientos con VoBo del PO, desarrollo, Pilot, Code Review y validación de QA.
2. **Improvements** lleva bugs y mejoras puntuales por un camino corto. `requirements` es opcional y solo aplica a mejoras.
3. Ambos kanbans terminan en `ready to merge`. **Cada semana** el RM integra esos tickets en la rama **Release Candidate (RC)**. Los tickets pasan a `merged` y **cierran en su propio kanban**.
4. El tablero **Releases** gestiona la **versión**, no los tickets. Hay una tarea de Release por producto y versión (la crea el SM) que recorre `dev preparation` → `unit test & ci` → `smoke` → `sanity` → `regression` → `ready for release` (Go/No-Go).
5. Tras el GO, **DevOps** despliega, QA valida en producción, el LD/RM monitorea y los incidentes abren el carril de **hotfix**.

---

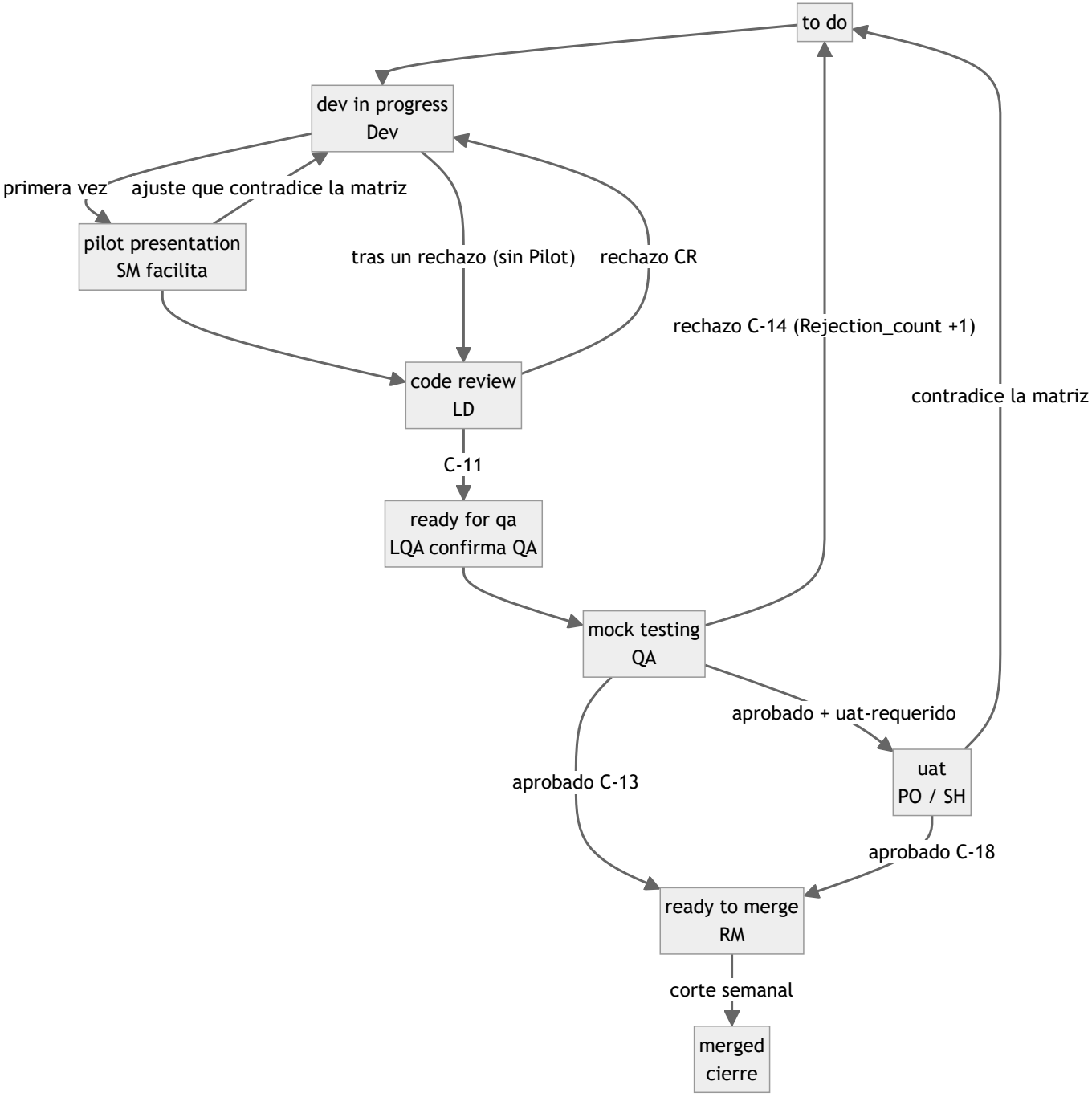
3. Discovery

3.1 Flujo y ciclo de rechazo

a) Definición (G1 a G4):



b) Construcción y validación (G5 a G8):



Reglas clave de transición:

- **Sin VoBo no se programa.** `requirements` → `to do` exige el comentario C-06 del PO y el tag `po-vobo` (automatización A-03).
- **Bypass de Pilot.** Un ticket corregido tras un rechazo **no** vuelve a `pilot presentation`: pasa de `dev in progress` a `code review`.
- `to do` **tiene dos usos:** cola para programar tras el VoBo y retorno por rechazo de QA.
- **UAT es condicional** y ocurre **antes** del merge (Q-16).
- `blocked` (transversal) puede usarse desde cualquier estado activo y vuelve al estado previo.
- `done w/d` es solo para **subtareas**. QA valida el ticket padre. Cuando el padre pasa a `ready to merge`, sus subtareas lo acompañan y lo siguen hasta `merged` (A-20, Q-53).

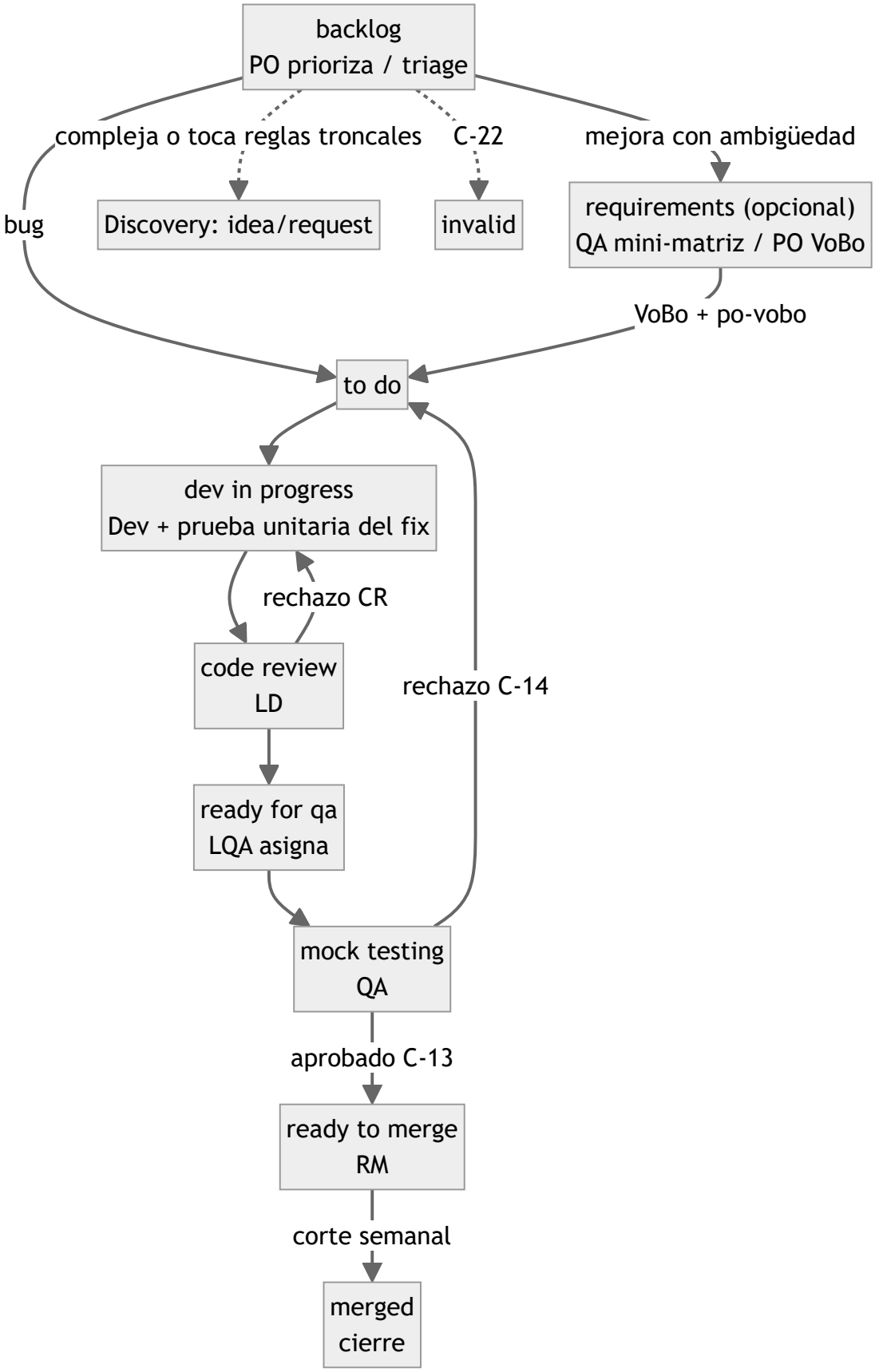
3.2 Tabla resumen

Estado	Owner (A)	Mueve	Reunión	Evidencia de salida	SLA / timebox
`idea/request`	PO	PO	SH + PO + LD + QA	Historia, valor y prioridad (C-01)	Siguiente Discovery Review
`assignment`	**LD** · LQA	LD / Dev	—	Dev como assignee, Dev Reviewer, Reviewer QA (lo asigna el LQA, Q-55) y tamaño (C-02)	1 día hábil
`r, d & architect review`	LD	Dev	Dev–LD puntual	Spike/POC aprobado (C-03)	S 1 · M 3 · L 5 días
`prototype review`	PO	PO / SM	**PO + Dev + QA + LD** (SM opc.)	Acta + contrato aprobado (C-04)	≤ 3 días para agendar
`requirements`	PO (VoBo) · QA (R)	QA	QA + Dev (PO si hay dudas)	Matriz vN + VoBo (C-05, C-06) + checklist de acuse del Dev	Matriz 1–3 días · VoBo 24 h
`to do`	Dev	Dev	—	—	Prioridad
`dev in progress`	Dev	Dev	Daily	PRs + handoff (C-09)	Estimación
`pilot presentation`	SM	SM	SH + PO + LD + QA + Dev	Acta y clasificación de ajustes (C-10)	Discovery Review
`code review`	LD	LD	Asíncrona (PR)	Aprobación (C-11)	1 día
`ready for qa`	LQA	QA	—	Reviewer QA confirmado + QA Points (C-12)	1 día
`mock testing`	QA	QA	QA + Dev puntual	C-13 / C-14 / C-17	QA Points
`uat`	PO	PO / SM	SH + Cliente + PO	C-18	≤ 3 días
`ready to merge`	RM	RM	—	Versión objetivo	Hasta el corte
`merged`	RM	RM	—	C-19	Durante `dev preparation`
`invalid`	PO	PO	—	Motivo (C-22)	—

3.3 Puntos de control (quality gates)

Gate	Entre	Condición para avanzar	Quién verifica
G1 Necesidad	`idea/request` → `assignment`	Historia + valor + prioridad + TYPE, Project, lond Subcategory	PO
G2 Solución	`r, d & architect review` → `prototype review`	Spike/POC aprobado por el LD	LD
G3 Prototipo	`prototype review` → `requirements`	Prototipo y contrato aprobados por PO, Dev, QA y LD	PO
G4 Requerimientos	`requirements` → `to do`	**Matriz con VoBo del PO** + checklist de acuse del Dev	QA / PO
G5 Demo	`pilot presentation` → `code review`	Visto bueno preliminar y ajustes clasificados	SM / PO
G6 Código	`code review` → `ready for qa`	PR aprobado y CI en verde	LD
G7 Calidad	`mock testing` → `ready to merge`	100 % de REQ críticos y 0 `urgent` abiertos	QA
G8 Aceptación	`uat` → `ready to merge`	Aprobación del PO/cliente (si aplica)	PO

4. Improvements



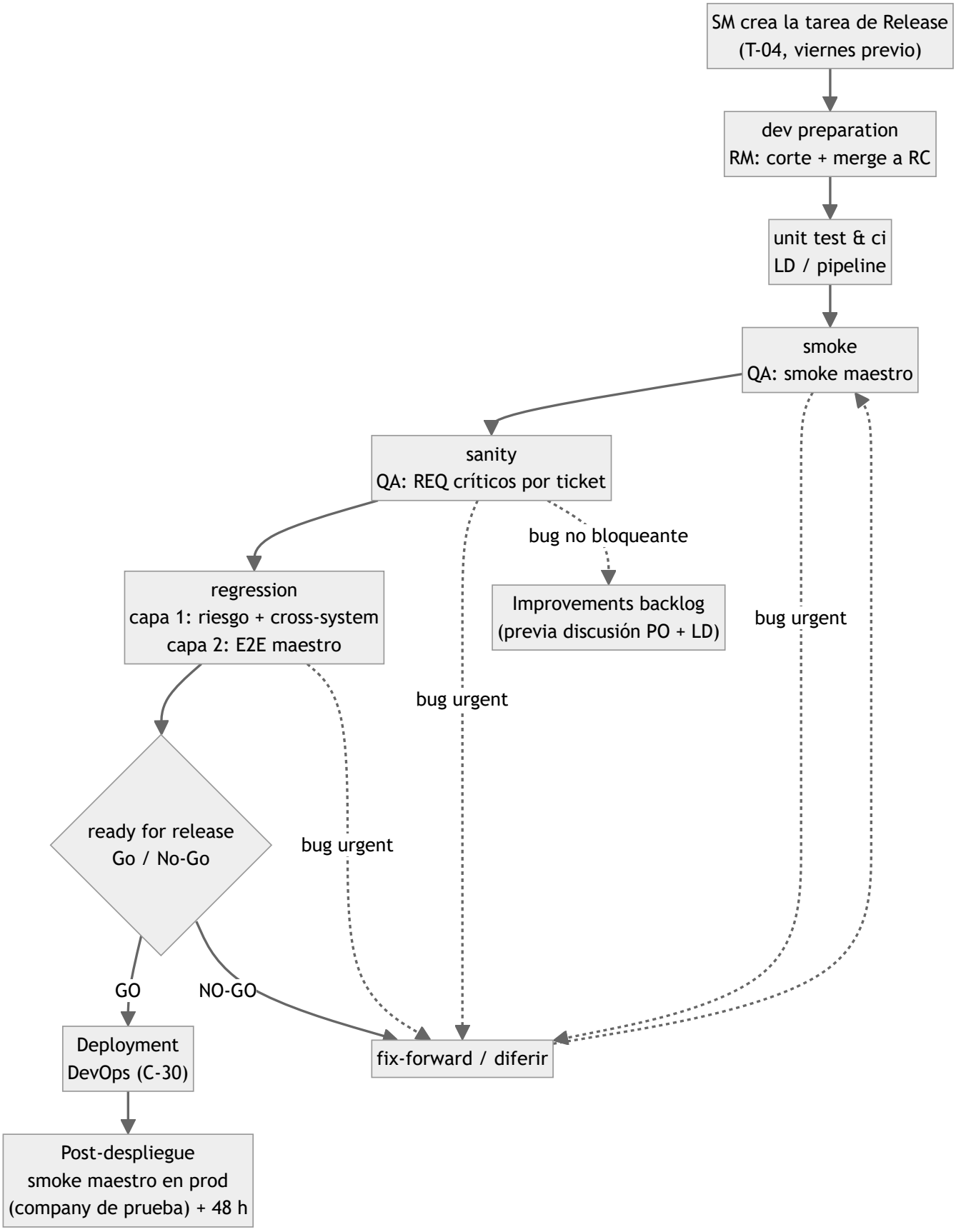
Estado	Owner (A)	Uso clave
`backlog`	PO (SM ordena)	Intake con plantilla T-02/T-03 (Form). Triage semanal: prioridad = severidad en bugs; decide si pasa por `requirements`.
`requirements`	PO (VoBo) · QA	**Opcional**: solo mejoras que necesitan criterios. Mini-matriz de ≤ 5 REQ. Los bugs no pasan.
`to do`	SM	Cola priorizada y retorno por rechazo.
`dev in progress`	Dev	Fix con prueba unitaria que certifica la solución.
`code review`	LD	PR aprobado y CI en verde.
`ready for qa`	LQA	Asignación del Reviewer QA.
`mock testing`	QA	Bug: resultado esperado + no regresión de zonas cercanas. Mejora: mini-matriz.
`ready to merge`	RM	Versión objetivo y corte semanal.
`blocked`	SM	Transversal, con causa `bloq-*`.
`done w/d`	Dev	Solo subtareas; siguen al padre a `ready to merge` y `merged`.
`merged` / `invalid`	RM / PO	Cierre / cierre negativo con motivo.

> **Sin `uat` en Improvements** (Q-30): en un caso muy excepcional que requiera validación del cliente, el PO decide si se valida en el `sanity` del release o si el ticket pasa a Discovery.

---

5. Releases

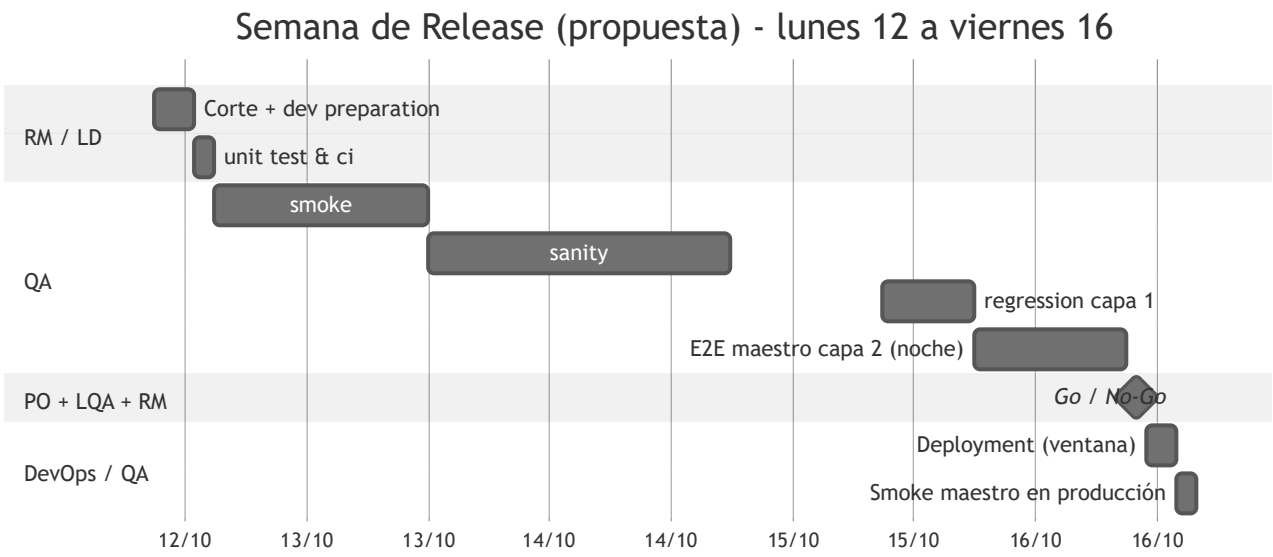
5.1 Flujo de la tarea de Release



Suites del release (Q-47 a Q-51): el **smoke maestro** es único y fijo para todo release (preprod y producción); el **sanity** y la **regresión capa 1** son propios de cada versión; el **E2E maestro** corre como **capa 2** dentro de `regression`: todo lo automatizado más un grupo de rotación manual (G1–G4), de modo que el maestro completo se cubre cada 4 releases. Detalle en la fuente canónica §7.7 y en el estándar de QA (doc 05 §6).

5.2 Calendario semanal (Q-32)

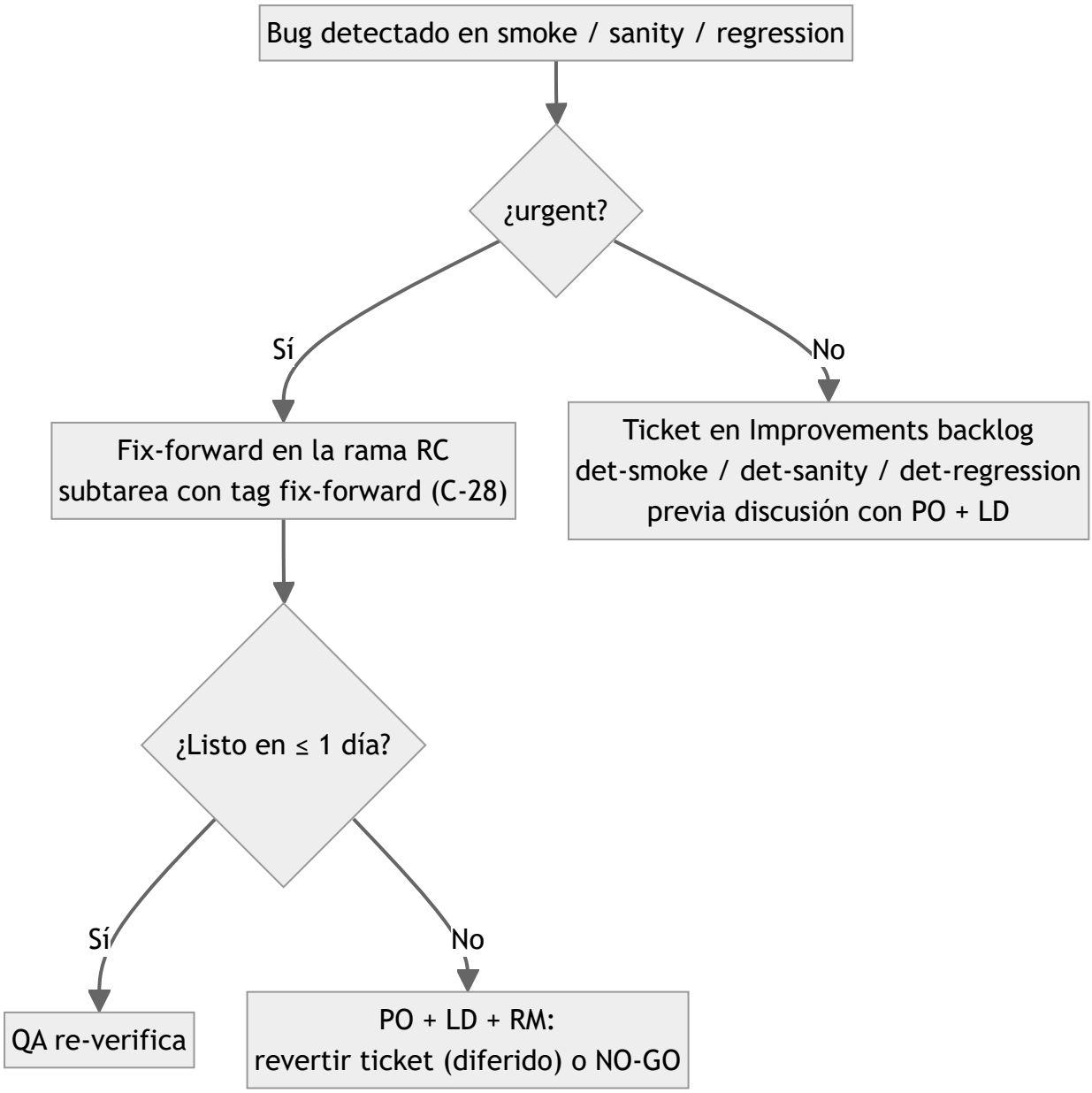
Las cajas de tiempo de la guía original (0,5 + 1 + 2 + 2 + 2 días) suman **7,5 días** y no caben en un sprint semanal. El calendario propuesto comprime las fases apoyándose en tres hechos: `sanity` **reutiliza** las matrices de `mock testing`, `regression` es **basada en riesgo** y el lote semanal es **pequeño**.



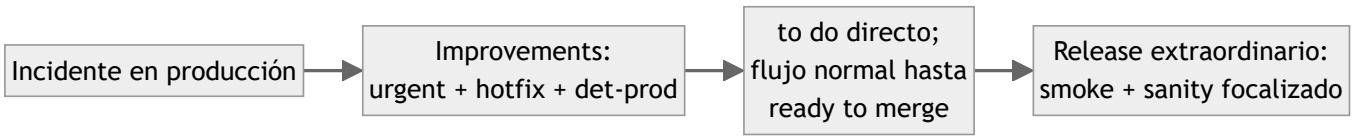
Día	Actividad	Owner	Timebox
Viernes previo	Candidatos (`ready to merge` + versión objetivo) y tarea de Release	SM	—
Lunes 09:00	**Corte**, `dev preparation` y `unit test & ci`	RM / LD	≤ 0,5 d
Lunes PM – martes AM	`smoke`	QA	≤ 0,5 d
Martes – miércoles	`sanity`	QA	≤ 1,5 d
Jueves	`regression`: capa 1 de día; capa 2 automatizada de noche y rotación manual; consolidado el viernes 09:00	QA	≤ 1 d + ≈ 0,5 d
Viernes 10:00	`ready for release` (Go/No-Go) y despliegue en la ventana acordada	PO + LQA + RM · DevOps	≤ 0,5 d

Reglas de corte (Q-32): sin tickets no hay release. Hay un límite de lote por capacidad de QA (QA Points). Los PRs de un ticket entran juntos o ninguno (merge atómico, Q-33).

5.3 Bugs durante el release y hotfix



Hotfix (incidente en producción):



5.4 Actividades posteriores al Go/No-Go (sin estado en ClickUp)

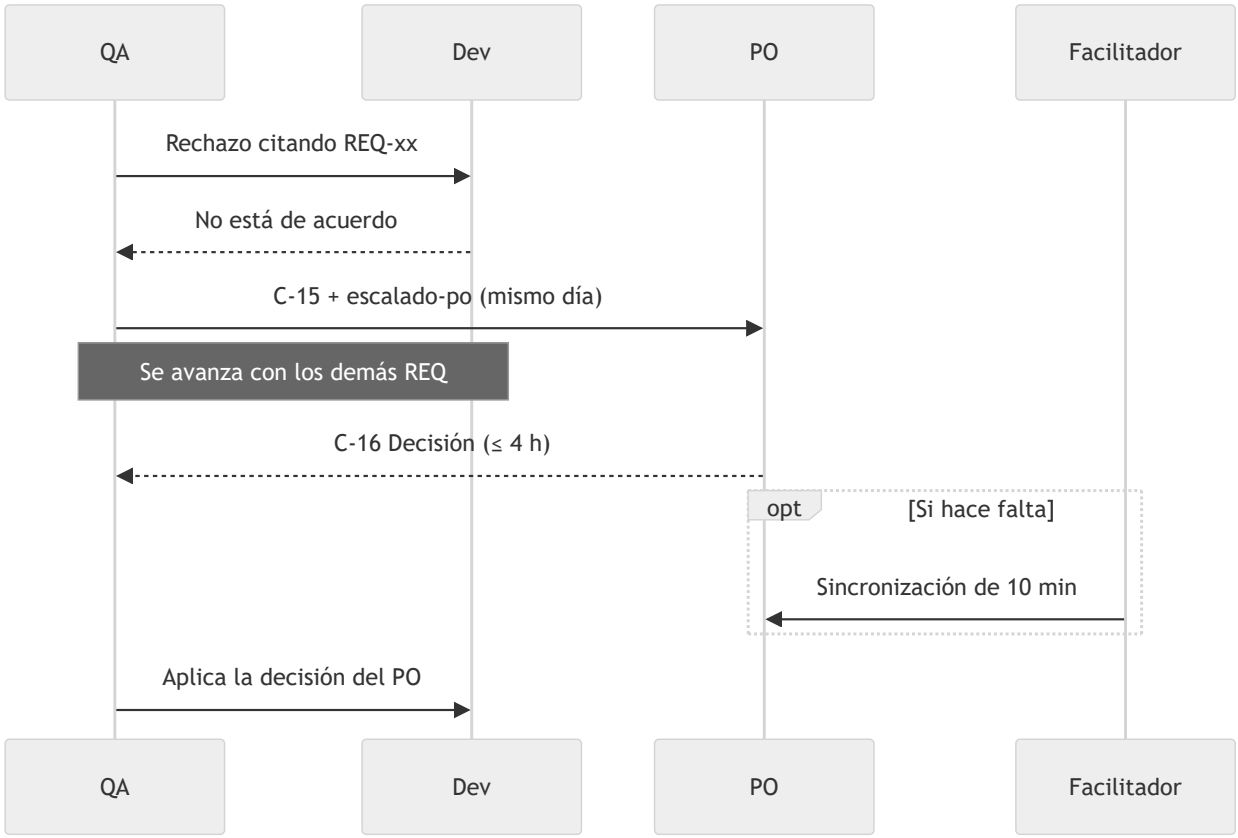
Actividad	Owner	Registro
Deployment	LD/RM coordina · **DevOps ejecuta** · el PO comunica a clientes	C-30 en la tarea de Release
Validación post-despliegue	QA (smoke maestro en producción con la company de prueba ≤ 2 h, con limpieza) · LD/RM con DevOps (48 h)	C-31
Exploratory / Bug bash	QA + Dev + SM (quincenal (Q-39))	`backlog` con `det-exploratorio`

Actividad	Owner	Registro
Validación continua en producción	PO + QA (reportes del soporte general de la empresa)	`hotfix` o `backlog` con `det-prod`

---

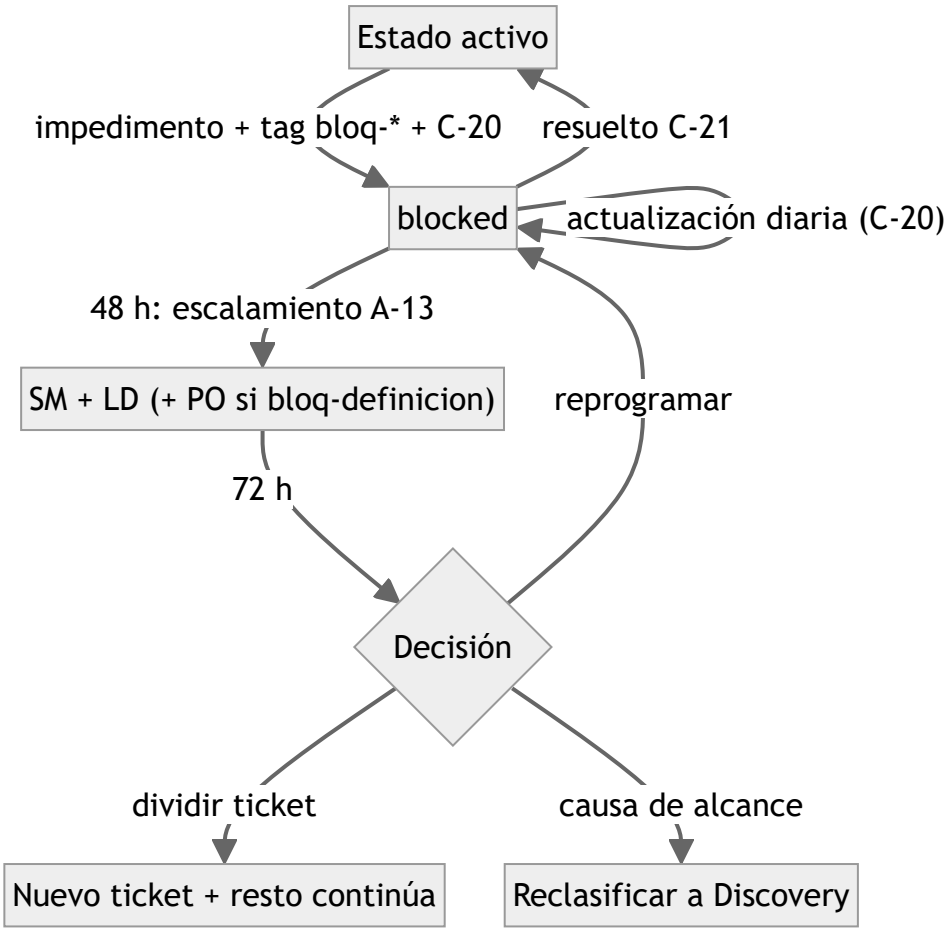
6. Reglas transversales

6.1 Escalamiento de desacuerdos



> Cuando el SM es parte del desacuerdo, **facilita el LD/RM** (Q-03). La decisión sobre los requerimientos sigue siendo del PO.

6.2 Bloqueos



6.3 Cambio de alcance después del VoBo

1. Solo lo solicita o aprueba el **PO**.
2. QA actualiza la matriz a **v+1** (C-08), se agrega `cambio-alcance` y se retira `po-vobo` (A-04).
3. El PO da un **nuevo VoBo** (C-06).
4. **Después** se modifica el código.

---

7. Matriz RACI por estado

R = Responsable (ejecuta) · **A** = Aprueba / rinde cuentas · **C** = Consultado · **I** = Informado.

Estado	PO	SM	LD	Dev	QA	LQA	RM
`idea/request`	**A/R**	C	C	I	C	I	—
`assignment`	I	I	**A/R**	R	I	R	—
`r, d & architect review`	I	I	**A**	**R**	C	—	—
`prototype review`	**A**	C	R	R	R	I	—
`requirements`	**A**	I	C	C	**R**	C	—
`to do`	I	R	I	**A**	I	—	—

Estado	PO	SM	LD	Dev	QA	LQA	RM
`dev in progress`	C	I	C	**A/R**	C	—	—
`pilot presentation`	A	**R**	C	R	C	—	—
`code review`	—	I	**A/R**	R	I	—	—
`ready for qa`	—	I	I	C	R	**A**	—
`mock testing`	C	I	I	C	**A/R**	C	—
`uat`	**A**	R	—	C	R	I	—
`ready to merge`	I	I	C	C	I	I	**A/R**
`merged`	I	I	I	I	I	I	**A/R**
`invalid`	**A/R**	I	I	I	I	—	—
`blocked`	C	**A/R**	C	R	C	—	—
`dev preparation`	I	C	C	C	I	I	**A/R**
`unit test & ci`	—	I	**A**	R	I	I	C
`smoke` / `sanity` / `regression`	I	I	C	C	**R**	**A**	C
`ready for release` (Go/No-Go)	**A**	R (facilita)	C	—	C	**A**	**A**
Deployment	I	I	C	—	I	I	**A** (DevOps R)

> Los Stakeholders (SH) son **C** en `idea/request`, `prototype review` y `pilot presentation` (pueden participar de forma asíncrona por video), y **R** en `uat`. En IOND, **LD y RM son la misma persona**: las columnas se mantienen separadas porque son responsabilidades distintas. Soporte no es un rol del equipo (se usa el soporte general de la empresa).

---

8. Contrato de frontera: QA ↔ PO ↔ Development

El nuevo rol de QA como Analista de Calidad y Requerimientos amplía su participación en la **definición**, pero **no** le transfiere la propiedad del producto. Esta tabla fija qué decide cada rol para evitar que los roles choquen.

Tema	PO	QA (Analista de Calidad y Requerimientos)	Development
Necesidad de negocio y prioridad	**Decide**	Analiza y pregunta	Informa la complejidad
Historia y criterios de alto nivel	**Redacta**	Revisa testabilidad	Revisa viabilidad
Matriz de Requerimientos	**Aprueba (VoBo único)**	**Redacta y mantiene**	Apoya técnicamente y acusa lectura
Reglas de negocio no definidas	**Decide**	Detecta y **propone** opción recomendada	Señala implicaciones técnicas; **no decide**
Casos fuera de alcance	Confirma en el VoBo	**Propone y documenta**	Usa como límite
Contrato de API / arquitectura	Aprueba en `prototype review`	Valida testabilidad	**Diseña** (LD aprueba)
Cambio de alcance	**Único que lo autoriza**	Versiona la matriz (v+1)	Implementa **después** del nuevo VoBo

Tema	PO	QA (Analista de Calidad y Requerimientos)	Development
Matriz de Pruebas	—	**Decide** (interna)	Puede consultarla
Aceptación o rechazo en `mock testing`	Resuelve desacuerdos	**Decide contra la matriz**	Corrige o escala
Hallazgos fuera de matriz	**Prioriza** el ticket nuevo	Registra (`fuera-de-matriz`)	—
UAT	**Decide si aplica** y aprueba	Acompaña y prepara datos	Soporta
Go/No-Go	**Vota** (riesgo de negocio)	LQA **certifica y vota** (calidad)	RM **vota** (operación)

Lo que QA no hace: priorizar el backlog, aprobar su propia matriz, decidir reglas de negocio, rechazar por preferencias no documentadas ni exigir que se repitan etapas ya aprobadas. **Lo que el PO no hace:** aprobar fila por fila, cambiar el alcance sin versionar la matriz, dejar reglas "a criterio del Dev". **Lo que Development no hace:** implementar algo distinto de lo acordado, decidir reglas de negocio ni cambiar el alcance sin el PO.

---

9. Artefactos por estado y dónde viven

Estado	Artefacto	Dónde vive (fuente oficial)	Copia para el catalyst (Fase F3)
`idea/request`	Historia + criterios (T-01)	Descripción del ticket	—
`r, d & architect review`	Spike/POC	Comentario C-03 + enlace a documento	—
`prototype review`	Acta + contrato aprobado	Comentario C-04 + enlace	Brief de arquitectura de QA
`requirements`	**Matriz de Requerimientos vN** + VoBo	Descripción (`## Matriz de Requerimientos vN`) + C-06	`requirements-matrix.md`
`dev in progress`	Handoff	Comentario C-09	—
`mock testing`	Matriz de Pruebas + reporte	C-13/C-14 + evidencias	`test-matrix.*` , `qa-report.md`
Release	Smoke / Sanity / Regression / Go-No-Go	Tarea de Release (C-23…C-31)	`releases/<producto>/<versión>/`

> **Regla:** ClickUp es la **fuente oficial**. Los archivos del catalyst son copias de trabajo que la IA usa como contexto y nunca sustituyen el registro en ClickUp.

Anexo A — Fichas completas por estado (fuente canónica)

5. Discovery — ficha por estado

Flujo feliz (primera vez): `idea/request` → `assignment` → `r, d & architect review` → `prototype review` → `requirements` → `to do` → `dev in progress` → `pilot presentation` → `code review` → `ready for qa` → `mock testing` → [`uat`] → `ready to merge` → `merged`

Ciclo de rechazo (QA): `mock testing` → `to do` → `dev in progress` → `code review` → `ready for qa` → `mock testing` (**no vuelve a pasar por** `pilot presentation`).

D-01 · `idea/request`

- **Tipo:** open · **Owner (A):** PO · **Quién mueve:** PO → `assignment` / `invalid`
- **Reunión:** obligatoria — SH + PO + LD + QA (refinamiento; se agrupa en la **Discovery Review** semanal, en un día dedicado que se acuerda con el SH — `Q-07`, AV-01).
- **Entrada:** ticket creado con la plantilla **T-01** y los campos TYPE, Project e Iond Subcategory (si Project = Iond).
- **Salida:** historia de usuario con criterios de aceptación de alto nivel, valor de negocio validado por SH y prioridad asignada. Comentario **C-01**.
- **SLA:** se refina en la siguiente *Discovery Review*.

D-02 · `assignment`

- **Tipo:** unstarted · **Owner (A): LD** (Dev) · **LQA** (QA) · **Quién mueve:** el LD asigna un Dev como **assignee** (`Q-46`) y el LQA asigna el **Reviewer QA** (`Q-55`); el Dev acepta y mueve a `r, d & architect review`.
- **Reunión:** no obligatoria.
- **Entrada:** request refinado y priorizado.
- **Salida:** Dev asignado (assignee), **Dev Reviewer** definido, **Reviewer QA** asignado por el LQA y tamaño estimado (S/M/L). El Dev confirma que tiene la información básica. Comentario **C-02**.
- **Por qué el QA se asigna aquí (`Q-55`):** el mismo QA sigue el R&D y el `prototype review`, redacta la matriz en `requirements` y ejecuta `mock testing`, sin traspasos de contexto.
- **SLA:** 1 día hábil.

D-03 · `r, d & architect review`

- **Tipo:** active · **Owner (A):** LD (aprueba) · **Responsable (R):** Dev · **Quién mueve:** Dev → `prototype review` una vez aprobado por el LD.
- **Reunión:** no obligatoria (sincronización técnica Dev–LD puntual).
- **Entrada:** Dev asignado.
- **Salida:** spike/POC documentado — opciones, recomendación, repos afectados, borrador del contrato de API, dependencias, riesgos y requisitos no funcionales — **aprobado por el LD** en comentario **C-03**. Los spikes se crean como **subtareas** de la funcionalidad y cierran en `done w/d` (`Q-14`).
- **Timebox (Q-08):** S ≤ 1 día · M ≤ 3 días · L ≤ 5 días.
- **QA (Reviewer QA):** sigue el R&D (C-03) para anticipar requisitos no funcionales, dependencias y datos de prueba (`Q-55`).
- **Si es inviable:** el PO decide `invalid` o replantear (`idea/request`).

D-04 · `prototype review`

- **Tipo:** active · **Owner (A):** PO (decide avanzar) · **Responsables:** Dev (presenta), LD (valida arquitectura), QA (testabilidad y captura inicial de requerimientos) · **SM:** opcional (facilita).
- **Reunión: obligatoria — PO + Dev + QA + LD** (SM opcional; SH convocados si aplica).
- **Entrada:** R&D aprobado; prototipo según el tipo de ticket (ver `Q-09`): wireframe/UX navegable, contrato de API (OpenAPI), diseño de board/flow y mapeo de datos, o dataset y criterio de calidad (IA).
- **Salida:** acta **C-04** con la aprobación formal del prototipo y del **contrato de interfaz**. El contrato aprobado pasa a formar parte de la **línea base** de la Matriz de Requerimientos (`Q-10`).

- **Siguiente:** OK → `requirements` · Errores de arquitectura o de concepto → `r, d & architect review`.

D-05 · `requirements`

- **Tipo:** active · **Owner (A):** PO (VoBo) · **Responsable (R):** QA (el Reviewer QA asignado en `assignment`, redacta) · **Consultados:** Dev, LD · **Quién mueve:** QA → `to do`, **solo con VoBo del PO** (comentario **C-06** + tag `po-vobo`).
- **Reunión:** no obligatoria (QA + Dev; PO si hay dudas).
- **Entrada:** acta de `prototype review`.
- **Salida:**
  - **Matriz de Requerimientos v1** en la descripción del ticket (formato en la sección 9), con su columna **Fuera de alcance**, los datos de prueba y mocks necesarios, y el marcado de REQ críticos.
  - Solicitud de VoBo **C-05**.
  - **Acuse del Dev:** ítem de checklist del ticket "*Matriz de Requerimientos vN leída por el Dev*", marcado por el Dev (`Q-23`). Las dudas se comentan con **C-07**.
  - VoBo del PO **C-06** + tag `po-vobo`.
- **SLA:** matriz en 1 día hábil (S/M) o 2–3 días (L) · VoBo del PO en 24 h hábiles · respuestas de QA a Dev en < 2 h hábiles.
- **VoBo que no llega (Q-17):** no hay aprobación tácita. A las 24 h QA vuelve a mencionar al PO y el SM lo plantea en la daily. A las 48 h el ticket pasa a `blocked` con `bloq-definicion`.
- **Siguiente:** VoBo → `to do` · Ajustes pedidos por el PO → permanece (matriz v+1) · Inviable → `prototype review`.

D-06 · `to do`

- **Tipo:** active · **Owner (A):** Dev · **SM:** prioriza la cola · **Quién mueve:** Dev → `dev in progress` al iniciar.
- **Entrada:** (a) primera vez: matriz con VoBo (`po-vobo`) y checklist de acuse marcado; (b) **retorno por rechazo de QA** (comentario **C-14**).
- **Salida:** el Dev toma el ticket.

D-07 · `dev in progress`

- **Tipo:** active · **Owner (A):** Dev · **Quién mueve:** Dev → `pilot presentation` (primera vez) o → `code review` (si viene de un rechazo).
- **Reunión:** Daily (Dev + SM).
- **Entrada:** ticket en `to do`.
- **Salida:**
  - Código según la matriz.
  - Pruebas unitarias en verde (meta de cobertura ≥ 80 % en código nuevo, si el repositorio lo soporta — `L-11`).
  - Linters limpios.
  - PR(s) registrados en **Merge Request 1–4**.
  - Comentario de handoff **C-09** (*Changes and Specifications Form for QA*).

D-08 · `pilot presentation`

- **Tipo:** active · **Owner (A):** SM (facilita y da seguimiento) · **Responsable (R):** Dev (presenta) · **Quién mueve:** SM → `code review`.
- **Reunión: obligatoria — SH + PO + LD + QA + Dev**. Se agrupa en la *Discovery Review* cuando sea posible.
- **Entrada:** funcionalidad terminada en el ambiente local/dev (solo la **primera vez**).
- **Salida:** acta **C-10** con el visto bueno preliminar del PO/SH. Tratamiento de ajustes (Q-11):

- **Contradice la matriz** → se corrige dentro del ticket (vuelve a `dev in progress`).
- **Es nuevo y el PO lo considera imprescindible** para el valor mínimo → cambio de alcance (matriz v+1, tag `cambio-alcance`, nuevo VoBo).
- **Es nuevo y no imprescindible** → ticket nuevo en Improvements con tag `pilot-feedback`, enlazado al padre.

D-09 · `code review`

- **Tipo:** active · **Owner (A):** LD · **Quién mueve:** LD (o Dev Reviewer) → `ready for qa` (aprobado) · → `dev in progress` (rechazado).
- **Reunión:** no obligatoria (asíncrona vía PR).
- **Entrada:** PR(s) en Merge Request 1–4; Dev Reviewer asignado.
- **Salida:** PR aprobado por al menos un LD o Senior (los PRs del LD/RM los revisan dos miembros con mayor antigüedad y **ambos deben aprobar**), CI en verde y análisis estático sin fallos (si existe). Comentario **C-11**.
- **SLA:** 1 día hábil.

D-10 · `ready for qa`

- **Tipo:** active · **Owner (A):** LQA · **Quién mueve:** el QA asignado → `mock testing` al iniciar.
- **Reunión:** no obligatoria (cola).
- **Entrada:** CR aprobado; build de la rama del ticket desplegado en el ambiente de QA, con migraciones y variables aplicadas. **Lo despliega el LD/RM con los workflows preparados por DevOps** (`Q-04`).
- **Salida:** el **Reviewer QA** asignado en `assignment` confirma los **QA Points** y la fecha de inicio; el LQA solo lo reasigna si la capacidad lo exige (`Q-55`). Comentario **C-12**.
- **SLA:** confirmación o reasignación en ≤ 1 día hábil.

D-11 · `mock testing`

- **Tipo:** active · **Owner (A):** QA · **Quién mueve:** QA → `uat` (con tag `uat-requerido`) / → `ready to merge` (aprobado) / → `to do` (rechazado; **Rejection_count + 1**).
- **Reunión:** no obligatoria (QA con soporte puntual del Dev).
- **Entrada:** ticket asignado; **Matriz de Pruebas** derivada de la Matriz de Requerimientos (trazabilidad REQ-ID → TC).
- **Salida:**
  - **100 % de los REQ críticos aprobados** y cero bugs `urgent` abiertos.
  - Comentario de aprobación **C-13** o de rechazo **C-14**, citando REQ-IDs y con evidencia.
  - Hallazgos fuera de la matriz → ticket nuevo con tag `fuera-de-matriz` (**no** bloquean). Comentario **C-17**.
- **Ambiente:** el **ambiente asignado al QA**, con la rama del ticket (`Q-05`). Si una integración no tiene sandbox, la sección *Datos de prueba* de la matriz indica cómo se prueba (mock, colección simulada o cuenta de prueba).

D-12 · `uat` (condicional)

- **Tipo:** Done (se ubica antes de `ready to merge`) · **Owner (A):** PO · **Responsables:** SH o cliente (validan), SM (coordina), QA (acompaña).
- **Reunión:** convocada — SH + Cliente + PO (+ QA).
- **Aplica solo si:** el ticket tiene el tag `uat-requerido` (lo marca el PO en `requirements`).
- **Entrada:** `mock testing` aprobado; rama del ticket desplegada en el ambiente de QA o preprod (Q-16).
- **Salida:** aprobación formal del PO/cliente en comentario **C-18** → `ready to merge`. Observaciones que contradicen la matriz → `to do` (bug). Observaciones nuevas → ticket nuevo.

- **SLA:** ≤ 3 días hábiles desde la convocatoria.

D-13 · `ready to merge`

- **Tipo:** Done · **Owner (A):** RM · **Quién mueve:** RM → `merged` durante `dev preparation` del release.
- **Entrada:** QA aprobado (y UAT si aplica); todos los PRs del ticket listos y sin conflictos.
- **Salida: versión objetivo** registrada según el Project (Iond.Version, Ionmind.Version, Stagemind.Version o G.Version). Entra al siguiente corte semanal. Sus subtareas en `done w/d` pasan también a `ready to merge` (**A-20**, `Q-53`).

D-14 · `blocked` (transversal)

- Ver la sección 8.3.

D-15 · `done w/d` (Done With Dependencies)

- **Tipo:** Done · **Uso exclusivo de subtareas** técnicas de una historia principal.
- **Owner (A):** Dev asignado a la subtarea · **Quién mueve:** Dev.
- **Entrada:** porción de código terminada, validada localmente e integrada en la rama del ticket padre.
- **Salida:** la subtarea se cierra **sin QA individual**; QA valida en el ticket padre. Cuando todas las subtareas cierran se notifica al owner del padre (**A-14**).
- **Sigue al padre (`Q-53`):** cuando el padre pasa a `ready to merge` , sus subtareas en `done w/d` pasan a `ready to merge` ; cuando el padre pasa a `merged` , ellas también (**A-20**). Si el padre vuelve a `to do` por un rechazo, las subtareas permanecen en `done w/d` hasta que el Dev reabra la que corresponda.
- **No usar** en tickets padre.

D-16 · `merged` (cierre)

- **Tipo:** Done (último estado del flujo) · **Owner (A):** RM · **Quién mueve:** RM.
- **Entrada:** PR(s) mergeados a la rama RC de la semana.
- **Salida:** comentario **C-19** con la rama RC, la versión y el enlace a la tarea de Release. Sus subtareas pasan también a `merged` (**A-20**). **El ticket queda cerrado en su kanban.**

D-17 · `invalid`

- **Tipo:** Closed · **Owner (A):** PO · **Quién mueve:** PO (o el SM por instrucción del PO).
- **Entrada:** decisión del PO (prioridad cambiada, requerimiento obsoleto o duplicado).
- **Salida:** comentario obligatorio **C-22** con el motivo exacto o el enlace al duplicado. Se cierra sin escribir código.

6. Improvements — ficha por estado

Bug: `backlog` → `to do` → `dev in progress` → `code review` → `ready for qa` → `mock testing` → `ready to merge` → `merged` **Mejora con criterios:** `backlog` → `requirements` → `to do` → … (igual que el bug) **Rechazo (QA):** `mock testing` → **`to do`** (+ Rejection_count)

I-01 · `backlog`

- **Tipo:** open · **Owner (A):** PO (prioriza) · **SM:** ordena y depura · **Registran:** QA, Dev, PO o el soporte general de la empresa, con la plantilla **T-02** (bug) o **T-03** (mejora), idealmente vía Form.
- **Reunión:** *Triage* semanal (PO + QA; LD si hace falta) (`Q-28`).
- **Clasificación (`Q-15`):** QA clasifica cada bug según su criterio (tablero, TYPE y prioridad = severidad) y el PO lo aprueba en el triage.
- **Entrada:** ticket con TYPE, Project, Iond Subcategory y prioridad (para bugs, la prioridad expresa la **severidad**: sección 10). Los bugs llevan el tag de detección `det-*` .
- **Salida:** priorizado y con decisión sobre si pasa por `requirements` (`Q-29`): bug → nunca; mejora → sí cuando el criterio no cabe en ≤ 3 líneas verificables; toca validaciones (cálculos, permisos, datos persistidos) o mensajes visibles para el usuario; o el TYPE es `New Feature` . Comentario **C-33**.

I-02 · `requirements` (opcional)

- **Tipo:** unstarted · **Owner (A):** PO (VoBo) · **Responsable (R):** QA (el LQA asigna el **Reviewer QA** al entrar — `Q-55`) · **Quién mueve:** QA → `to do` con `po-vobo` .
- **Reunión:** opcional (QA + Dev + LD).
- **Salida: mini-matriz** (≤ 5 REQ, con su columna Fuera de alcance), checklist de no regresión de zonas cercanas y checklist de acuse del Dev (`Q-23`). Comentarios **C-05**, **C-06** y tag `po-vobo` .
- **SLA:** QA 1 día hábil · VoBo 24 h hábiles.
- **Escalamiento:** si resulta compleja o toca reglas troncales → el PO reclasifica a Discovery.

I-03 · `to do`

- **Tipo:** unstarted · **Owner (A):** SM (priorización ágil) · **Quién mueve:** Dev → `dev in progress` .
- **Entrada:** priorizado (y con VoBo si pasó por `requirements`) **o** devuelto por rechazo de QA.

I-04 · `dev in progress`

- **Tipo:** active · **Owner (A):** Dev · **Reunión:** Daily.
- **Salida:** fix o mejora implementado, **prueba unitaria que certifica la solución del defecto**, PR en Merge Request, handoff **C-09** (versión corta).

I-05 · `code review`

- **Tipo:** active · **Owner (A):** LD · **Salida:** PR aprobado y CI en verde → `ready for qa` · rechazo → `dev in progress` . Comentario **C-11**.

I-06 · `ready for qa`

- **Tipo:** active · **Owner (A):** LQA · **Salida:** Reviewer QA asignado por el LQA (o confirmado, si la mejora pasó por `requirements` — `Q-55`); build en el ambiente de QA. Comentario **C-12**.

I-07 · `mock testing`

- **Tipo:** active · **Owner (A):** QA.
- **Criterio de aprobación:**
  - **Bug:** se cumple el resultado esperado del reporte y pasa el checklist de no regresión de zonas cercanas.

- **Mejora:** mini-matriz aprobada.
- **Siguiente:** aprobado → `ready to merge` (**C-13**) · rechazo → `to do` (**C-14**, Rejection_count + 1).

I-08 · `ready to merge`

- **Tipo:** Done · **Owner (A):** RM · **Salida:** versión objetivo registrada; entra al corte semanal. Las subtareas en `done w/d` lo acompañan (**A-20**).

I-09 · `blocked` (transversal)

- Ver la sección 8.3.

I-10 · `done w/d`

- Igual que **D-15** (solo subtareas; siguen al padre a `ready to merge` y `merged`).

I-11 · `merged` (cierre)

- **Tipo:** Done (último estado del flujo) · **Owner (A):** RM · **Salida:** PR mergeado a la rama RC; comentario **C-19**; subtareas a `merged` (**A-20**). El ticket queda cerrado en Improvements.

I-12 · `invalid`

- **Tipo:** Closed · **Owner (A):** PO · **Salida:** comentario obligatorio **C-22**.

> **Sin `uat` en Improvements (`Q-30`):** si un caso muy excepcional requiere validación del cliente, el PO decide si se valida en el `sanity` del release o si el ticket pasa a Discovery.

7. Releases — tarea de Release, estados, calendario y reglas

7.1 Modelo

- Cada semana existe una lista de sprint en el folder **Releases**: `Sprint N (fecha inicio - fecha fin)`.
- El **SM crea una tarea de Release por producto y versión** — `Release <producto> - vX.Y.Z` — con la plantilla **T-04** y la verificación del Dev. La tarea documenta: alcance, tickets incluidos (relación con los tickets y lista), repos y ramas RC, ambiente, prerrequisitos, smoke, plan de regresión, hallazgos y Go/No-Go.
- **Los tickets no se mueven al tablero de Releases**: permanecen en su kanban y cierran en `merged`. La tarea de Release los referencia por relación y por **versión objetivo** (Iond.Version, Ionmind.Version, Stagemind.Version o G.Version, según el Project).
- Si en el corte no hay tickets en `ready to merge` para un producto, **no se crea release** esa semana (Q-32).
- **Datos mínimos de la tarea de Release** (para métricas): **fecha de vencimiento** = viernes del Go/No-Go, **Project**, **versión** (Iond.Version, Ionmind.Version, Stagemind.Version o G.Version) y **relación** con cada ticket incluido. El resultado del Go/No-Go se escribe en **C-29** con el texto normalizado `Decisión: GO | CONDITIONAL GO | NO-GO` (en la plantilla se deja una sola opción; la lectura automática ignora negritas y emoji).
- Todo bug `det-prod` o `hotfix` se **relaciona** con la tarea de Release de la versión donde se detectó (cálculo del change failure rate).

7.2 Estados de la tarea de Release

#	Estado	Tipo	Owner (A)	Qué ocurre	Salida	Timebox (Q-32)
REL-01	`dev preparation`	open	**RM**	Corte; merge de los tickets en `ready to merge` (versión objetivo) a la rama RC de cada repo; conflictos resueltos; despliegue a preprod; tickets → `merged`.	Ramas RC creadas, builds OK, preprod actualizado y prerrequisitos verificados. Comentario **C-23**.	≤ 0,5 d (lunes AM)
REL-02	`unit test & ci`	active	LD (pipeline) · Dev	Ejecución automática de las pruebas unitarias de los repos de la RC. Si fallan, bloquean el pipeline.	100 % en verde. Comentario **C-24**.	Horas (lunes)
REL-03	`smoke`	active	QA	**Smoke maestro** (`Q-47`): la misma lista fija SMK-01…SMK-17 en todo release (login SSO, dashboard, crear y ejecutar un board en test y producción, historial, listados clave, admin y health de iond, IonMind y StageMind). No incluye verificaciones de la versión: esas van en `sanity`.	Smoke OK (**C-25**). Si falla en el core: fix-forward o No-Go.	≤ 0,5 d (lunes PM – martes AM)
REL-04	`sanity`	active	QA	**Sanity de la versión** (`Q-48`): re-ejecución de los **REQ críticos** de la matriz de cada ticket del release en el ambiente integrado (preprod), más las interacciones entre tickets del mismo módulo o Iond Subcategory.	Todos los tickets validados en conjunto sin efectos colaterales (**C-26**).	≤ 1,5 d (martes–miércoles)
REL-05	`regression`	active	QA	**Capa 1 — regresión de la versión** (`Q-49`): suite basada en riesgo seleccionada del E2E maestro (baseline crítico + módulos impactados + cross-system iond ↔ IonMind ↔ StageMind ↔ Gateway/ShipEdge Core; extendida si el riesgo es Alto). **Capa 2 — E2E maestro** (`Q-50`): la parte automatizada completa, sin supervisión, y el grupo de rotación manual que corresponde (G1–G4). Usa bot-test donde exista.	Capa 1: flujos críticos al 100 %. Capa 2: fallas analizadas y clasificadas. **C-27** con ambas capas.	Capa 1 ≤ 1 d (jueves) · capa 2 jueves noche, resultados el viernes antes de las 09:00
REL-06	`ready for release`	closed	**LQA** (certifica)	Mesa de decisión **Go/No-Go** — reunión obligatoria **PO + LQA + RM** (Q-35) al cerrar `regression`. La tarea pasa a `ready for release` solo con GO o CONDITIONAL GO; con NO-GO permanece en `regression` (Q-35).	Decisión GO / CONDITIONAL GO / NO-GO (**C-29**), release notes, plan de rollback verificado y ventana de despliegue con DevOps.	≤ 0,5 d (viernes AM)

7.3 Actividades posteriores (sin estado en ClickUp; se registran como comentario en la tarea de Release)

Código	Actividad	Owner	Salida
ACT-01	**Deployment**	LD/RM coordina · **DevOps ejecuta** · el PO comunica a clientes	Despliegue ejecutado; comentario **C-30** con fecha/hora, versiones y repos.
ACT-02	**Validación post-despliegue**	QA (smoke maestro en producción con la **company de prueba**, ≤ 2 h, `Q-52`) · LD/RM con DevOps (monitoreo 48 h)	Comentario **C-31** (incluye la limpieza de los datos creados); los incidentes abren `hotfix`.
ACT-03	**Exploratory testing / Bug bash**	QA + Dev + SM	Sesión quincenal de 60–90 min (Q-39); hallazgos → `backlog` con `det-exploratorio`.
ACT-04	**Validación continua en producción**	PO + QA (con los reportes del soporte general de la empresa)	Incidentes → `hotfix` o `backlog` con `det-prod`.

7.4 Calendario semanal (Q-32)

Día	Actividad
Viernes previo	El SM verifica los candidatos (tickets en `ready to merge` con versión objetivo) y prepara la tarea de Release.
Lunes 09:00	**Corte.** `dev preparation` (RM) + `unit test & ci` . Smoke por la tarde.
Martes	Smoke (cierre AM) + Sanity.
Miércoles	Sanity.
Jueves	Regression capa 1 (día) · E2E maestro capa 2 automatizado (noche) · rotación manual del grupo que corresponde.
Viernes 09:00	QA consolida la capa 2 en el C-27.
Viernes 10:00	Go/No-Go. Despliegue en la ventana acordada con DevOps (Q-36). Post-despliegue: smoke maestro en producción con la company de prueba.

7.5 Bugs encontrados durante el release

1. **Bug bloqueante (`urgent`)** → **fix-forward** en la misma rama RC. Se registra como subtarea de la tarea de Release con el tag `fix-forward` (comentario **C-28**); el Dev corrige, el LD revisa y QA re-ejecuta la verificación afectada.
2. **Resto de bugs** → según su prioridad, **tickets nuevos en Improvements** (`backlog`, tag `det-smoke` / `det-sanity` / `det-regression`), previa discusión con el **PO y el LD**.
3. Si el fix-forward no está listo en el plazo acordado (Q-34: decisión ≤ 4 h, fix ≤ 1 día): el PO, el LD y el RM deciden entre **revertir el ticket de la RC** (vuelve a `to do` con el tag `diferido`) o **NO-GO**.

7.6 Hotfix (incidentes en producción) (Q-38)

Se reporta un incidente (soporte general de la empresa, clientes o monitoreo) → ticket en Improvements (TYPE Bug, prioridad `urgent`, tags `hotfix` + `det-prod`) → `to do` directo (sin backlog) → `dev in progress` → `code review` → `ready for qa` → `mock testing` → `ready to merge` → **Release extraordinario** `Hotfix <producto> - vX.Y.Z+1` (con tag `hotfix`) en el sprint vigente (smoke + sanity focalizado) → despliegue. Lo autorizan el PO y el LD; lo ejecuta el RM.

7.7 Suites de prueba del release (`Q-47` a `Q-51`)

Suite	Alcance	Fuente	Ambiente	Comentario	Peso en el Go/No-Go
Smoke maestro	Única y fija para todo release (SMK-01…SMK-17)	`knowledge/test-suites/smoke-master.md`	Preprod (gate) y producción con la company de prueba	C-25 · C-31	Core fallido = fix-forward o NO-GO
Sanity	Propia de la versión: REQ críticos de los tickets del release	Matrices de Requerimientos y de Pruebas de cada ticket	Preprod	C-26	100 % de los REQ críticos
Regression capa 1	Propia de la versión, por riesgo	E2E maestro + mapa Iond Subcategory → módulo + riesgo de la T-04	Preprod	C-27 (capa 1)	100 % de los flujos críticos

Suite	Alcance	Fuente	Ambiente	Comentario	Peso en el Go/No-Go
E2E maestro capa 2	Todas las features: automatizado completo + rotación manual G1–G4 (maestro completo cada 4 releases)	`knowledge/test-suites/e2e-master.csv` (IDs estables; la hoja "lond Testing Plan" es la fuente editable)	Preprod	C-27 (capa 2)	Falla = hallazgo; NO-GO solo si es `urgent` provocado por el release

Los hallazgos de cualquier suite siguen la sección 7.5 (tags `det-smoke`, `det-sanity`, `det-regression`).

8. Reglas transversales de transición

8.1 Rechazo

- **Discovery:** `mock testing` → `to do` → `dev in progress` → `code review` → `ready for qa` → `mock testing`. **Bypass de Pilot:** ningún ticket corregido tras un rechazo vuelve a `pilot presentation`.
- **Improvements:** `mock testing` → `to do`.
- **Code review rechazado:** → `dev in progress` (no cuenta en Rejection_count).
- Cada rechazo de QA: comentario **C-14** (cita los REQ-ID incumplidos) + **Rejection_count + 1**.
- En el re-test, QA verifica las correcciones **sin exigir repetir etapas ya aprobadas** (Pilot, definiciones).

8.2 Criterio de rechazo

- QA rechaza **únicamente** contra la Matriz de Requerimientos con VoBo vigente (incluido el contrato aprobado en `prototype review` cuando la matriz lo referencia — `Q-10`).
- Lo no contemplado **no** es rechazo directo: es un ticket nuevo con `fuera-de-matriz` (**C-17**) y el PO decide la prioridad.
- Implementar algo distinto de lo acordado **sí** es motivo de rechazo.

8.3 `blocked` (transversal)

- **Entra:** quien detecta el impedimento, desde cualquier estado activo. Tag de causa obligatorio: `bloq-externo`, `bloq-ambiente`, `bloq-definicion` o `bloq-dependencia`. Comentario **C-20** con el plan de acción. Si aplica, se usa la dependencia nativa de ClickUp (*waiting on*).
- **Seguimiento:** actualización diaria del plan (**C-20**) en la daily (Dev + LD + SM + QA).
- **48 h** → escalamiento automático (**A-13**) a SM + LD (+ PO si es `bloq-definicion`).
- **72 h** → decisión: (a) reprogramar, (b) dividir el ticket, (c) reclasificar a Discovery **solo** si la causa es de alcance o estructura (decide el PO).
- **Sale:** vuelve al estado previo con el comentario **C-21**. El tag `bloq-*` se conserva como historial (métricas de bloqueo por causa).

8.4 Desacuerdos (QA ↔ Dev)

1. Registrar en el ticket el punto exacto y la postura de cada parte en 1–2 líneas (**C-15**) y agregar el tag `escalado-po`.
2. Notificar al PO el mismo día (automatización **A-15**).
3. El PO decide (**C-16**, SLA 4 h hábiles) y se retira `escalado-po`.

4. Si hace falta, sincronización de 10 minutos facilitada por el SM. Si el SM es parte del desacuerdo, **facilita el LD/RM** (`Q-03`).
5. La decisión queda registrada en el ticket.

8.5 Cambio de alcance (después del VoBo)

1. Solo lo solicita o aprueba el **PO**.
2. QA actualiza la matriz a la versión v+1 y marca las filas cambiadas (**C-08**). Se agrega el tag `cambio-alcance` y se retira `po-vobo` (**A-04**). El tag `cambio-alcance` se conserva como historial; si el ticket ya lo tenía, QA retira `po-vobo` manualmente.
3. El PO vuelve a dar su VoBo (**C-06**) y **luego** se modifica el código.

8.6 `invalid`

- Solo el PO decide. Comentario obligatorio **C-22**.

8.7 Corte de transición (Q-45)

- Tickets que aún no llegaron a `dev in progress` adoptan el flujo completo (Matriz de Requerimientos + VoBo).
- Tickets en `dev in progress` o más adelante terminan con su documentación vigente y siguen el resto de reglas (rechazo, `merged` , Release).